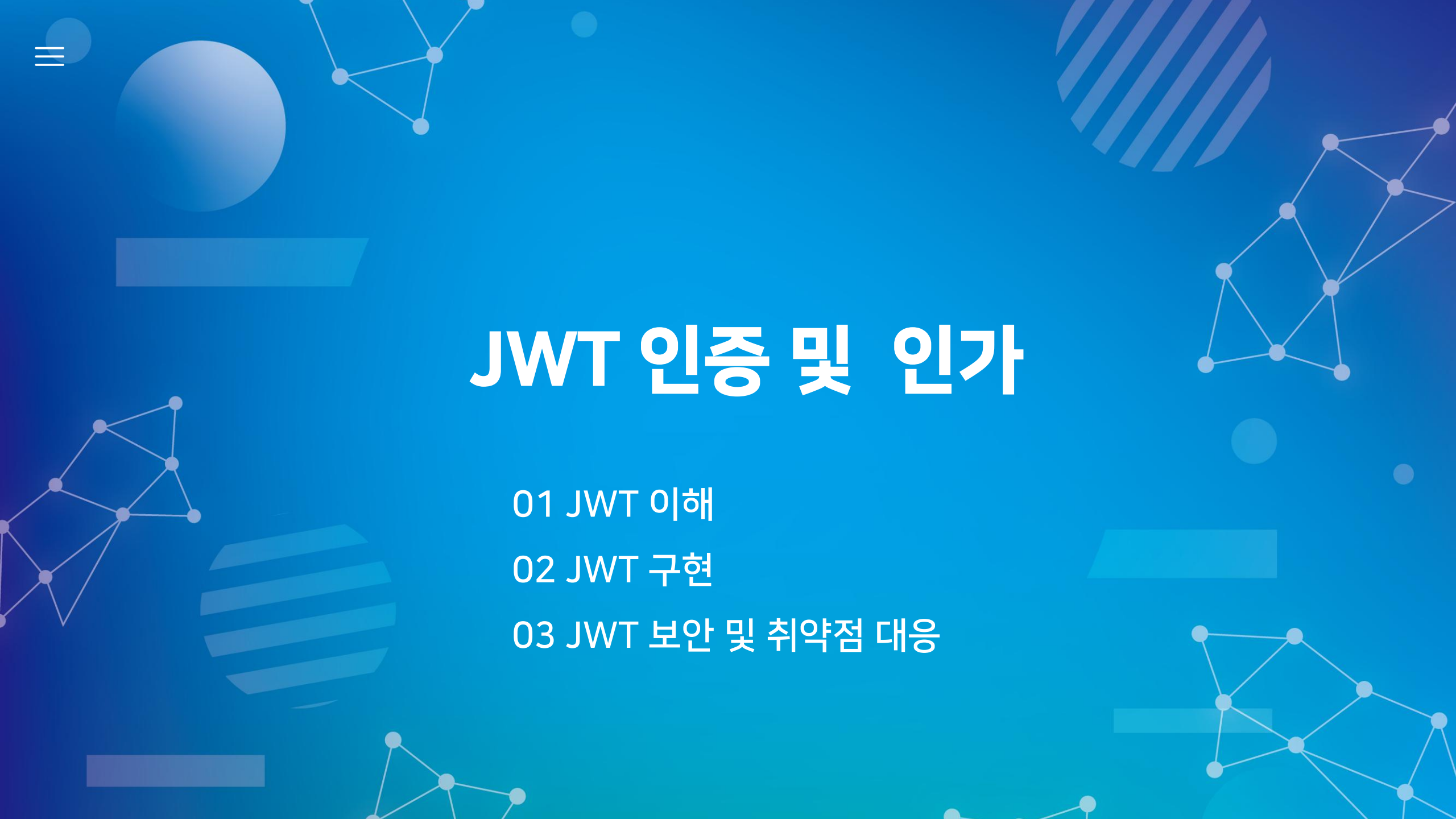


The background is a vibrant blue gradient. It features several abstract geometric elements: a large solid blue sphere in the upper left, a smaller sphere with diagonal stripes in the upper right, and another striped sphere in the lower left. There are also several network diagrams consisting of interconnected nodes and lines, scattered across the slide. A small white hamburger menu icon is located in the top left corner.

JWT 인증 및 인가

01 JWT 이해

02 JWT 구현

03 JWT 보안 및 취약점 대응



학습 목표

- ✅ JWT 개념과 필요성을 이해하고 설명할 수 있다.
- ✅ JWT 기반 인증 흐름을 이해하고, 구현할 수 있다.
- ✅ JWT 보안 위협을 이해하고 이에 대한 해결 방안을 설명할 수 있다.



학습 내용

1. JWT 개념

2. JWT 구현

3. JWT 보안 위협 및 해결방법



JWT 인증 및 인가

01 JWT 이해

02 JWT 구현

03 JWT 보안 및 취약점 대응

JWT(JSON Web Token) 이란?

- JWT란?

JSON Web Token은 JSON 형식의 데이터를 안전하게 전송하기 위한 개방 표준(RFC 7519)입니다.

주로 인증(Authentication)과 인가(Authorization)에 사용되며, 클라이언트-서버 간 정보를 교환하는 용도로 활용됩니다.

- 주요 특징

- 자체 포함(Self-contained): 토큰 내에 사용자 정보, 만료 시간 등의 정보를 포함하여 별도 저장소 없이 인증 가능
- 경량(Compact): URL, 헤더, 쿠키 등 다양한 환경에서 전송하기 적합한 크기
- 서명(Signature): 토큰의 위변조 여부를 확인하기 위해 HMAC, RSA, ECDSA 등의 알고리즘으로 서명

JWT의 구성 및 동작 방식

- JWT 구성 요소

- 헤더(Header)

토큰의 타입(JWT)과 서명에 사용할 알고리즘 정보(예: HS256, RS256) 포함

- 페이로드(Payload)

- 인증 관련 정보(클레임, claims) 포함

- 표준 클레임: iss(발급자), exp(만료시간), sub(주체), aud(대상) 등

- 커스텀 클레임: 애플리케이션별 추가 데이터

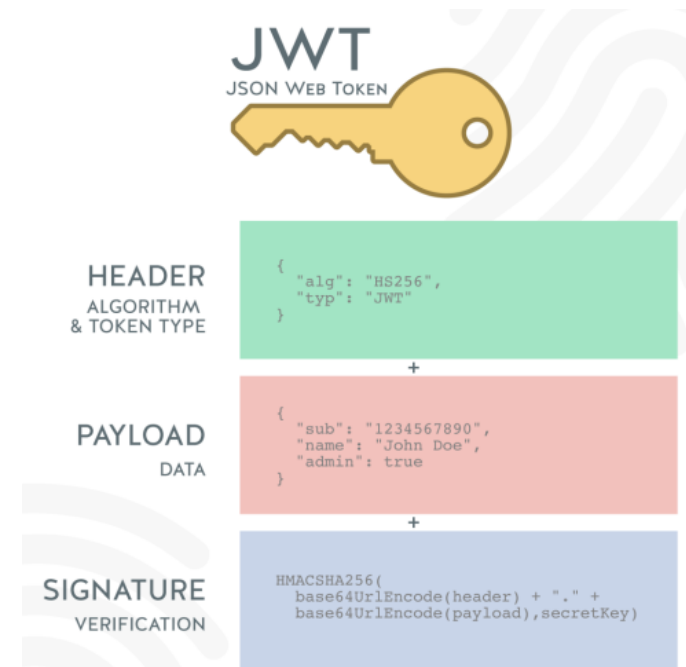
- 서명(Signature)

헤더와 페이로드를 비밀 키로 서명하여 위변조 방지

- 동작 원리

클라이언트는 인증 후 JWT를 발급받아 이후 요청할 때마다 포함

서버는 JWT의 서명을 검증하여 토큰의 유효성을 확인한 후, 페이로드 내 정보를 기반으로 권한을 부여



출처: <https://blog.knoldus.com/json-web-token-jwt-a-complete-guide/>

JWT의 장점

- 무상태성 (Stateless & Self-contained)

서버에 별도의 세션 저장소가 필요 없어, 확장성과 분산 시스템에 적합

- 빠른 검증 및 성능

JWT는 자체에 모든 정보를 포함하므로, 데이터베이스 조회 없이도 신원 검증 가능

- 유연한 클레임 구조

표준 클레임 외에도 애플리케이션 맞춤형(claims) 데이터를 포함할 수 있어 다양한 정보 교환이 용이

- 표준 기반

RFC 7519에 의해 정의되어 다양한 플랫폼과 언어에서 지원 및 호환성 보장

JWT의 단점

- 정보 노출 문제

기본적으로 Base64 인코딩되어 있으므로, 암호화하지 않으면 누구나 페이로드 내용을 확인할 수 있음

- 토큰 탈취 위험

토큰 자체에 중요한 정보가 포함되어 있으며, 만료 기간 동안 노출 시 악용 가능 (HTTPS 등 전송 보안 필요)

- 토큰 무효화의 어려움

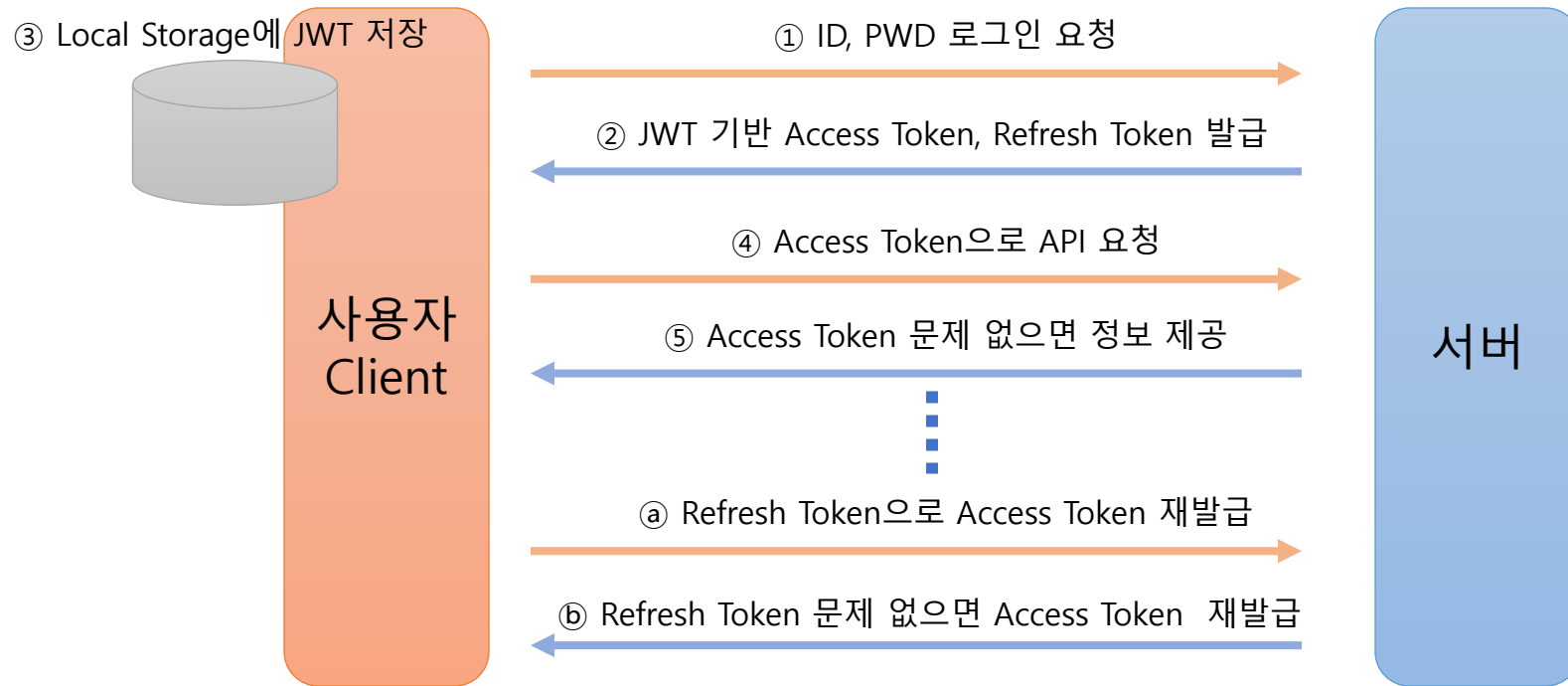
발급된 JWT는 만료 전까지 유효하므로, 탈취 시 빠른 무효화가 어렵고 재발급 관리가 복잡

- 크기 및 오버헤드

여러 클레임을 포함할 경우 토큰 크기가 커져 전송 오버헤드가 발생할 수 있음

JWT 기반 인증 흐름

JWT를 이용한 로그인 및 액세스 토큰 발급 과정



JWT 기반 인증 흐름

- JWT를 활용한 사용자 인가 및 권한 관리

- 토큰 내 클레임(Claims) 활용

- JWT 페이로드에 사용자 식별자, 역할(role), 권한(permission) 등의 정보를 포함

- 클라이언트는 인증 후 JWT를 발급받아, 이후 요청 시 HTTP 헤더에 포함 ("Bearer <token>")

- 서버 측 검증 및 인가 과정

- 서버는 JWT의 서명, 만료시간, 발급자 등을 검증하여 토큰의 무결성 확인

- 페이로드의 클레임을 기반으로 사용자의 접근 권한 및 인가 결정

- 예) 특정 API에 접근하기 위해 필요한 역할 또는 권한이 JWT 클레임에 포함되어 있는지 확인

- 무상태(Stateless) 인증

- 서버는 별도의 세션 저장소 없이, JWT 내 정보를 활용하여 인가를 수행

- 분산 시스템 및 마이크로서비스 환경에서 효과적

- 토큰 만료 및 갱신

- 짧은 만료 시간 설정으로 권한 변경 시 빠른 반영 가능

- Refresh Token과 토큰 회전 전략을 통해, 오래된 권한 정보를 가진 토큰 사용 방지

JWT 기반 인증 흐름

- JWT를 활용한 사용자 인가 및 권한 관리 사용시 고려사항

- 동적 권한 관리의 한계 및 보완

사용자 권한 변경 시, 기존 JWT에 반영되지 않으므로 주기적 토큰 갱신 또는 별도 인가 체크 필요
중앙 인가 서버와의 연계를 통해 실시간 권한 업데이트 구현 고려

- 보안 강화

HTTPS를 통한 안전한 전송, HttpOnly/ Secure 쿠키 등 안전한 저장소 활용
최소한의 정보만 포함하고, 민감 정보는 별도 암호화 또는 안전한 관리



JWT 인증 및 인가

01 JWT 이해

02 JWT 구현

03 JWT 보안 및 취약점 대응

JWT 구현 실습

- hello-spring-boot 프로젝트에 기능 추가하여 실습

Spring Boot 기본 사용법 설명에서 실습하였던 프로젝트 활용

Java 17 / Maven 환경

- 실습 내용

- Access Token과 Refresh Token의 생성
- Access Token의 검증
- Refresh Token을 이용하여 새로운 Access Token과 Refresh Token 재발급

프로젝트 환경 설정

- pom.xml 의 Dependency 목록에 JWT 라이브러리(JJWT)를 추가함
 - jjwt-api : 관련 API 정의
 - jjwt-impl : 실제 API 동작 구현
 - jjwt-Jackson : Jackson 라이브러리 연동으로 JSON 처리

```
28     </scm>
29     <properties>
30         <java.version>17</java.version>
31     </properties>
32     <dependencies>
33         <dependency>
34             <groupId>org.springframework.boot</groupId>
35             <artifactId>spring-boot-starter-web</artifactId>
36         </dependency>
37         <dependency>
38             <groupId>io.jsonwebtoken</groupId>
39             <artifactId>jjwt-api</artifactId>
40             <version>0.11.5</version>
41         </dependency>
42         <dependency>
43             <groupId>io.jsonwebtoken</groupId>
44             <artifactId>jjwt-impl</artifactId>
45             <version>0.11.5</version>
46             <scope>runtime</scope>
47         </dependency>
48         <dependency>
49             <groupId>io.jsonwebtoken</groupId>
50             <artifactId>jjwt-jackson</artifactId>
51             <version>0.11.5</version>
52             <scope>runtime</scope>
53         </dependency>
54     </dependencies>
55
56     <build>
57         <plugins>
58             <plugin>
```

JWT 컨트롤러 소스코드 (1/4)

- 상수값 설정

- secretKey : HS256 암호화 비밀 키
- ACCESS_TOKEN_EXPIRATION : 10분
- REFRESH_TOKEN_EXPIRATION : 30분

```
1 package com.example.hellospringboot.controller;
2
3 import io.jsonwebtoken.Claims;
4 import io.jsonwebtoken.Jwts;
5 import io.jsonwebtoken.SignatureAlgorithm;
6 import io.jsonwebtoken.security.Keys;
7 import org.springframework.http.ResponseEntity;
8 import org.springframework.web.bind.annotation.GetMapping;
9 import org.springframework.web.bind.annotation.RequestMapping;
10 import org.springframework.web.bind.annotation.RequestParam;
11 import org.springframework.web.bind.annotation.RestController;
12
13 import javax.crypto.SecretKey;
14 import java.util.Date;
15 import java.util.HashMap;
16 import java.util.Map;
17
18 @RestController
19 @RequestMapping("/jwt")
20 public class JwtController {
21
22     // HS256 알고리즘에 적합한 안전한 비밀키 생성
23     private final SecretKey secretKey = Keys.secretKeyFor(SignatureAlgorithm.HS256);
24
25     // 토큰 만료 시간 (Access: 10분, Refresh: 30분)
26     private final long ACCESS_TOKEN_EXPIRATION = 10 * 60 * 1000;
27     private final long REFRESH_TOKEN_EXPIRATION = 30 * 60 * 1000;
28 }
```

JwtController.java

JWT 컨트롤러 소스코드 (2/4)

- 엔드포인트 구현

- /jwt/generate : Access Token, Refresh Token 생성

- Access Token 생성 과정

- setSubject("test-user"): 사용자 식별 정보
 - setIssuer("example.com"): 토큰 발급자
 - setIssuedAt(new Date()): 발급 시각
 - setExpiration(new Date(...)): 만료 시간 설정
 - claim("role", "user"): 추가 클레임 (예: 사용자 권한)
 - signWith(secretKey): 비밀키로 서명

- Refresh Token 생성

기본 정보 외에 claim("tokenType", "refresh") 추가로 토큰 구분

```
28
29  /**
30   * Access Token과 Refresh Token을 함께 생성하여 반환합니다.
31   * URL: /jwt/generate
32   */
33  @GetMapping("/generate")
34  public Map<String, String> generateTokens() {
35      // Access Token 생성
36      String accessToken = Jwts.builder()
37          .setSubject("test-user")
38          .setIssuer("example.com")
39          .setIssuedAt(new Date())
40          .setExpiration(new Date(System.currentTimeMillis() + ACCESS_TOKEN_EXPIRATION))
41          .claim("role", "user")
42          .signWith(secretKey)
43          .compact();
44
45      // Refresh Token 생성 (tokenType 클레임을 추가하여 구분)
46      String refreshToken = Jwts.builder()
47          .setSubject("test-user")
48          .setIssuer("example.com")
49          .setIssuedAt(new Date())
50          .setExpiration(new Date(System.currentTimeMillis() + REFRESH_TOKEN_EXPIRATION))
51          .claim("tokenType", "refresh")
52          .signWith(secretKey)
53          .compact();
54
55      Map<String, String> tokens = new HashMap<>();
56      tokens.put("accessToken", accessToken);
57      tokens.put("refreshToken", refreshToken);
58      return tokens;
59  }
60
```

JwtController.java

JWT 컨트롤러 소스코드 (3/4)

· 엔드포인트 구현

/jwt/verify : Access Token 유효성 검증

```
61-  /**
62-   * 전달받은 Access Token의 유효성을 검증하고 클레임 정보를 반환합니다.
63-   * URL: /jwt/verify?token={토큰}
64-   */
65-  @GetMapping("/verify")
66-  public ResponseEntity<?> verifyAccessToken(@RequestParam String token) {
67-      try {
68-          Claims claims = Jwts.parserBuilder()
69-              .setSigningKey(secretKey)
70-              .build()
71-              .parseClaimsJws(token)
72-              .getBody();
73-
74-          return ResponseEntity.ok(claims);
75-      } catch (Exception e) {
76-          return ResponseEntity.badRequest().body("Invalid Token: " + e.getMessage());
77-      }
78-  }
79-  }
```

JwtController.java

JWT 컨트롤러 소스코드 (4/4)

· 엔드포인트 구현

/jwt/refresh : Refresh Token이 유효할 경우 새로운 Access Token과 Refresh Token을 발급

이전 Refresh Token 재사용을 방지하려면 서버 측에서 DB나 캐시를 활용해야 함

```
80 /**
81  * Refresh Token을 사용하여 새로운 Access Token과 Refresh Token을 발급합니다.
82  * URL: /jwt/refresh?refreshToken={RefreshToken}
83  */
84 @GetMapping("/refresh")
85 public ResponseEntity<> refreshAccessToken(@RequestParam String refreshToken) {
86     try {
87         // 최신 방식의 파서를 사용하여 Refresh Token 검증
88         Claims claims = Jwts.parserBuilder()
89             .setSigningKey(secretKey)
90             .build()
91             .parseClaimsJws(refreshToken)
92             .getBody();
93
94         // tokenType 클레임을 확인하여 실제 Refresh Token인지 검증
95         String tokenType = (String) claims.get("tokenType");
96         if (tokenType == null || !tokenType.equals("refresh")) {
97             return ResponseEntity.badRequest().body("Invalid Token type !");
98         }
99
100         // (선택 사항) DB나 캐시 등에서 기존 토큰 상태를 확인하여 이전 토큰 재사용을 방지하는 로직 추가 가능
101
102         // 새로운 Access Token 생성
103         String newAccessToken = Jwts.builder()
104             .setSubject(claims.getSubject())
105             .setIssuer("example.com")
106             .setIssuedAt(new Date())
107             .setExpiration(new Date(System.currentTimeMillis() + ACCESS_TOKEN_EXPIRATION))
108             .claim("role", "user")
109             .signWith(secretKey)
110             .compact();
111
```

```
112         // Refresh Token Rotation: 새로운 Refresh Token 발급
113         String newRefreshToken = Jwts.builder()
114             .setSubject(claims.getSubject())
115             .setIssuer("example.com")
116             .setIssuedAt(new Date())
117             .setExpiration(new Date(System.currentTimeMillis() + REFRESH_TOKEN_EXPIRATION))
118             .claim("tokenType", "refresh")
119             // 필요 시 고유 토큰 ID 부여 로직 추가 (ex. .setId(newUniqueTokenId()))
120             .signWith(secretKey)
121             .compact();
122
123         Map<String, String> tokens = new HashMap<>();
124         tokens.put("accessToken", newAccessToken);
125         tokens.put("refreshToken", newRefreshToken);
126         return ResponseEntity.ok(tokens);
127     } catch (Exception e) {
128         return ResponseEntity.badRequest().body("Invalid Refresh Token: " + e.getMessage());
129     }
130 }
131 }
```

JwtController.java

구현된 JWT 기능 테스트 (1/3)

- Access Token 발급 시험

실제 환경에서는 사용자 인증 성공한 사용자에게 대해서만 처리되어야 함

웹 브라우저에서 `http://localhost:8080/jwt/generate` 입력

JSON 형식으로 `accessToken`, `refreshToken` 값이 반환됨

파이어폭스 브라우저의 경우 결과 화면에서 '원시값 보기' 선택

```
{"accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXZXQ", "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXZXQ"}
ILMNVDSISIMIndC16MTc0MTA3MzM3MCwiZXhwIjozODc0OTcwLCJyY2x1IjoiaXNlciJ9.dpP1yQ
Xho7dtZmGoDBoeizGNGtQ7g05RJSike1vcpwY", "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXZXQ
dWIiOiJ0ZXN0LXVzZXIiLCJpc3MiOiJleGFtcGxILMNVDSISIMIndC16MTc0MTA3MzM3MCwiZXhwIjoz
NzQxMDc1MTcwLCJ0b2tlblR5cGUioiJyZWZyZXNoIn0.mLjZs-
iZ9xH67IoCm7F_Ktie6LmdMsUaU_8B59VEuB8"}
```

구현된 JWT 기능 테스트 (2/3)

- Access Token 유효 확인 시험

Access Token의 유효성 및 클레임 내용 확인

웹 브라우저에서 `http://localhost:8080/jwt/verify?token=액세스토큰값` 입력

토큰이 유효할 경우 JSON 형식으로 토큰 안에 들어있는 클레임 정보 반환됨

잘못된 토큰일 경우 HTTP 400 응답으로 에러 메시지가 반환됨

```
{"sub":"test-user","iss":"example.com","iat":1741073370,"exp":1741073970,"role":"user"}
```

< 토큰이 유효할 경우 >

```
Invalid Token: JWT signature does not match locally computed signature. JWT validity cannot be asserted and should not be trusted.
```

< 잘못된 토큰일 경우 >

구현된 JWT 기능 테스트 (3/3)

- ## · Refresh Token 으로 Access Token 및 Refresh Token 회전 시험

Refresh Token의 유효성 확인 후 재발급 처리

웹 브라우저에서 `http://localhost:8080/jwt/refresh?refreshToken=리프레시토큰값` 입력

토큰이 유효할 경우 JSON 형식으로 새로운 Access Token과 Refresh Token이 반환됨

잘못된 토큰일 경우 HTTP 400 응답으로 에러 메시지가 반환됨

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiI0ZXN0LXVzZXIiLCJpc3MiOiIjleGFtcGx1LnMvbnBIsImlddCI6MTc0MTA3NDg1MCwiZXBwIjoibG9jaW5kaW8weC1ybnQSLmQ4P2bmahakpDGe9zmV_M-DgCtsON9HuOp8q5M",
  "refreshToken": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiI0ZXN0LXVzZXIiLCJpc3MiOiIjleGFtcGx1LnMvbnBIsImlddCI6MTc0MTA3NDg1MCwiZXBwIjoibG9jaW5kaW8weC1ybnQSLmQ4P2bmahakpDGe9zmV_M-DgCtsON9HuOp8q5M"}
}
```

< 토큰이 유효할 경우 >

Invalid Refresh Token: JWT signature does not match locally computed signature. JWT validity cannot be asserted and should not be trusted.

< 잘못된 토큰일 경우 >



JWT 인증 및 인가

01 JWT 이해

02 JWT 구현

03 JWT 보안 및 취약점 대응

JWT 보안 및 취약점 대응 - 서명과 키 관리

· JWT 서명의 역할

토큰의 무결성과 진위 보증: 헤더와 페이로드가 변조되지 않았음을 확인

토큰 위변조 방지를 위한 디지털 서명 적용

· 서명 알고리즘 종류

- 대칭키 알고리즘: HS256, HS384, HS512
- 비대칭키 알고리즘: RS256, RS384, RS512, ES256 등

헤더	페이로드	서명
{ "alg": "HS256", "typ": "JWT" }	{ "sub": "1234567890", "name": "Mickey", "iat": 125334534 }	HMACSHA256(base64UrlEncode(header) + "." + base64UrlEncode(payload), 256-bit-secret)

JWT 구조

· 서명 과정

헤더와 페이로드를 Base64 URL 인코딩 → 비밀 키(또는 개인키)로 서명 생성 → 최종 JWT 구성

JWT 보안 및 취약점 대응 - 서명과 키 관리

- 알고리즘 혼동 공격

공격자가 "none" 알고리즘을 이용하거나 다른 알고리즘으로 위장하여 서명 우회 시도
(대응 방안)

허용 알고리즘을 명시적으로 설정하고, 라이브러리의 알고리즘 검증 강화

- 비밀 키/개인 키 유출

키가 노출되면 토큰 위조가 가능하여 인증 우회 및 권한 탈취 위험 증가
(대응 방안)

강력한 키 생성 및 안전한 저장(HSM, 클라우드 키 관리 서비스)

키 접근 권한 최소화 및 보안 정책 강화

- 키 교체 및 회전 관리 미흡

키 회전 시 기존 토큰 검증 문제, 만료되지 않은 토큰이 악용될 위험
(대응 방안)

정기적인 키 회전 정책과 토큰 재발급, JWKS 엔드포인트 활용

JWT 보안 및 취약점 대응 - 유효기간 관리

- JWT 유효기간의 역할

만료 시간(exp) 클레임: JWT 내에 포함되어 토큰의 유효 기간을 명시
만료 시간을 통해 탈취된 토큰의 사용 기간을 제한하여 피해 최소화

- 유효기간 설정의 중요성

너무 긴 유효기간은 토큰 탈취 시 장기간 악용 위험 증가
너무 짧은 유효기간은 사용자 경험 저하 및 잦은 재인증 요구
서비스 특성에 맞게 적절한 유효기간 판단 필요

- 유효기간 관리 전략

짧은 만료 시간으로 설정한 Access Token 사용하면서 상대적으로 긴 만료 시간의 Refresh Token을 통한 재발급 전략 사용

JWT 보안 및 취약점 대응 - JWT 탈취

- 주요 JWT 탈취 경로

- XSS 공격

- 웹 애플리케이션의 입력값 검증 미흡 취약점을 악용하여 악성 스크립트 삽입을 통해 JWT를 탈취

- 불안정한 저장소

- 로컬 스토리지나 세션 스토리지에 JWT를 저장할 경우, 악성 코드에 의해 유출 위험 존재

- 전송 중 탈취

- HTTPS 미적용 또는 취약한 TLS 설정 시 네트워크 스니핑 공격에 노출

- 위험 요소 요약

- JWT 탈취 시, 탈취된 토큰으로 정상 사용자의 권한을 도용하여 시스템에 접근할 수 있음

- 대응 방안

- 충실한 XSS 방어

- 안전한 저장소 사용

- 전송 보안 확보



요약 정리

- JWT는 인증과 인가를 위한 강력한 도구지만, 서명 검증, 유효기간 관리, 키 보호가 필수적.
- JWT는 암호화되지 않으므로 민감한 데이터를 직접 포함하면 안 됨.
- Refresh Token을 활용한 인증 유지 전략을 적용하고, 토큰 탈취 방지를 위한 XSS 방어 및 안전한 저장소 사용이 중요.



참고 문헌

- [KISA] 소프트웨어 개발 보안 가이드(2021.12.29)
- [KISA] 소프트웨어 보안약점 진단가이드(2021)
- [오픈이지북스] 실무에 바로 적용하는 해킹 방어를 위한 JAVA 시큐어코딩
- [홍릉과학출판사] 소프트웨어 개발 보안 실무 - 시큐어 코딩 원리와 실제
- [secu BOOK] 웹 해킹과 시큐어코딩 탐지/수정 실습가이드
- [에이콘출판사] 보안 위협 모델링 Threat Modeling - 위협 식별과 대응을 위한 소프트웨어 설계
- [책만] 모두를 위한 소프트웨어 보안 설계와 구현 - 취약점 분석부터 위협 모델링과 완화 전략, 보안 패턴, 암호화까지 안전한 소프트웨어의 모든 것
- [위키북스] 웹 애플리케이션 보안 완벽 가이드 - 체계적으로 배우는 안전한 웹 애플리케이션 제작 기법
- [한빛미디어] 웹 애플리케이션 보안 - 정찰, 공격, 방어 세 단계로 배우는 웹 애플리케이션 보안의 모든 것
- [한빛미디어] 웹 해킹 & 보안 완벽 가이드 - 웹 애플리케이션 보안 취약점을 겨냥한 공격과 방어

The background is a vibrant blue gradient. It is decorated with various abstract elements: a large solid blue sphere in the upper left, a smaller sphere with diagonal stripes in the upper right, and another striped sphere in the lower left. Several network diagrams, consisting of interconnected nodes and lines, are scattered across the image. There are also various rectangular and trapezoidal shapes in different shades of blue. In the top left corner, there is a small icon of three horizontal lines.

감사합니다.



퀴즈

4) 헤더-페이로드-트레일러 로 구성

4) 과도한 크기로 전송 오버헤드 발생 가능성

$$\left(\begin{array}{c} \text{ } \\ \text{ } \\ \text{ } \end{array} \right)$$



퀴즈 정답

1. JWT의 주요 특징에 해당하지 않는 것은?

4) 헤더-페이로드-트레일러 로 구성

JWT는 헤더-페이로드-서명 으로 구성되어 있는 토큰 형식입니다.

2. JWT의 단점 중 틀린 것은?

1) 만료 시간 전 강제 무효화를 위한 관리자 페이지 필요

JWT는 기본적으로, 만료 기간 전에 강제로 만료할 수 있는 방법이 없습니다.

3. JWT 페이로드에 포함시키는 사용자 식별자, 역할, 권한 등의 정보를 일컫는 단어는?

(클레임 또는 claim)